

Optymalizacja problemu Multiple Sequence Alignment (MSA) z wykorzystaniem wielowątkowej metaheurystyki Tabu Search

Oskar Kałuziak

10 czerwca 2026

Spis treści

1	Treść problemu	3
2	Wstęp i cel	4
3	Wielowątkowość i Architektura Hybrydowa	5
3.1	Równoległy Multi-Start (Parallel Multi-Start) (C++)	5
3.2	Protokół komunikacji z interfejsem	5
4	Algorytm Tabu Search	7
4.1	Reprezentacja rozwiązania	7
4.2	Funkcja celu (Dual-Objective Pipeline)	7
4.3	Generowanie sąsiedztwa	7
4.4	Lista Tabu i Kryterium Aspiracji	9
4.5	Heurystyka Naprawcza (Procedura Repair)	9
4.6	Zaawansowane Mechanizmy Dywersyfikacji	10
4.6.1	Strategic Oscillation (Heartbeat)	10
4.6.2	Heavy Diversification (Kick-Move)	10
5	Złożoność Obliczeniowa i Pamięciowa	11
5.1	Złożoność czasowa funkcji celu	11
5.2	Złożoność generowania sąsiedztwa i pętli głównej	11
5.3	Złożoność procedury naprawczej	11
5.4	Narzut pamięciowy (Space Complexity)	12
6	Interfejs użytkownika (UI) i Generator Instancji	13
6.1	Generator instancji i wprowadzanie szumu	13
6.2	Konfiguracja i parametryzacja procesów obliczeniowych C++	13
6.3	Wizualizacja zbieżności i raportowanie	14

7	Testy i Wyniki Eksperymentalne	20
7.1	Metodologia badań i definicja miar	20
7.2	Skalowalność i analiza odporności na szum (Mutacje)	20
7.3	Optymalizacja parametrów metaheurystyki	22
7.3.1	Wpływ dywersyfikacji wielowątkowej (Liczba wątków T)	22
7.3.2	Rozmiar generowanego sąsiedztwa (k)	23
7.4	Złożoność problemu a rozmiar użytego alfabetu	24
8	Wnioski i kierunki dalszych prac	26
8.1	Analiza wydajności i stabilności	26
8.2	Perspektywy rozwoju algorytmu	26

1 Treść problemu

Problem *Multiple Sequence Alignment* (MSA), czyli dopasowanie wielu sekwencji, jest jednym z podstawowych problemów w bioinformatyce. W wersji klasycznej polega on na tym, że z otrzymanej macierzy zawierającej m ciągów znaków (sekwencji) o początkowych długościach n konieczne jest dopasowanie ich tak, aby zmaksymalizować ich podobieństwo w kolumnach. Dopasowanie jest wykonywane poprzez wstawianie do sekwencji znaków pustych, tzw. luk (najczęściej reprezentowanych znakiem spacji lub w tym przypadku znakiem podkreślenia $_$). Powstanie luk symuluje zjawiska biologiczne takie jak insercje czy delecje.

W poniższym projekcie rozpatrywany jest bardziej restrykcyjny wariant tego problemu, określany w czasopiśmie jako **MIN LENGTH ALIGNMENT**. Problem definiuje się następująco: dla danego zbioru sekwencji wejściowych (m sekwencji o długości n) zbudowanych ze skończonego alfabetu (np. $\Sigma = \{A, C, G, T\}$), celem jest znalezienie takiego dopasowania, aby spełniony został **twardy warunek braku konfliktów** w kolumnach. W praktyce oznacza to, że w żadnej kolumnie macierzy wyrównania nie mogą wystąpić dwie różne litery alfabetu. Innymi słowy, nukleotyd 'A' nie może znajdować się w tej samej kolumnie co nukleotyd 'C'. Luki ($_$) są ignorowane podczas takowego sprawdzania konfliktów w kolumnach macierzy wyrównania. Funkcją celu, którą chcemy zminimalizować, jest całkowita długość macierzy dopasowania (liczba kolumn macierzy, licząc także wstawione przerwy). Optimum stanowi zatem najkrótsze możliwe bezkonfliktowe ułożenie sekwencji. Ponieważ problem ten jest silnie NP-trudny, znalezienie optymalnego globalnie dopasowania dla dużych m oraz n jest możliwe jedynie za pomocą wydajnych algorytmów przybliżonych (metaheurystyk).

2 Wstęp i cel

Głównym celem projektu było opracowanie metaheurystyki opartej na algorytmie **Tabu Search** do rozwiązywania problemu *MIN LENGTH ALIGNMENT* wraz z jej implementacją oraz badaniami eksperymentalnymi. Przyjęto hybrydową architekturę aplikacji, dzieląc system na dwa główne moduły:

1. **Wielowątkowy moduł obliczeniowy (C++)**: Główny moduł algorytmiczny projektu. Zaimplementowany w C++ z naciskiem na maksymalizację wydajności i równoległe wykonywanie obliczeń. Opiera się na metaheurystyce (**Tabu Search**) z dedykowanymi operatorami sąsiedztwa (przesunięcie bloków, modyfikacje kolumn luk) oraz specjalizowaną heurystyką naprawczą (**napraw()**), która zapewnia, że zwracane rozwiązania są całkowicie bezkonfliktowe.
2. **Interfejs graficzny i generator instancji (Python)**: Warstwa wizualna jak i operacyjna zbudowana została w oparciu o Tkinter w Pythonie. Odpowiada ona za generowanie wymagających, poprawnych instancji problemu (poprzez wycinanie podciągów z nadciągu o długości d). Dodatkowo stworzona została opcja dodawania szumu (mutacji) oraz wizualizacji zbieżności funkcji celu w czasie rzeczywistym za pomocą biblioteki Matplotlib.

Poniższe sprawozdanie szczegółowo omawia zawarte w projekcie rozwiązania, przedstawia wzór funkcji celu i kar zastosowanych w projekcie. Opisuje techniki dywersyfikacji przeszukiwania (tzw. *Heartbeat* oraz *Kick-Move*) oraz zawiera rozległy rozdział obejmujący eksperymenty pozwalające zbadać wpływ parametrów wejściowych na jakość oraz czas działania zaproponowanej metody.

3 Wielowątkowość i Architektura Hybrydowa

Z uwagi na dużą złożoność obliczeniową problemu *MIN LENGTH ALIGNMENT* oraz silnie NP-trudną naturę przeszukiwanej przestrzeni zastosowano hybrydową architekturę. Intefejs realizowany jest w języku Python, natomiast ciężkie obliczenia matematyczne zostały przeniesione do niskopoziomowego modułu algorytmicznego w C++. Zaimplementowana została również pełna wielowątkowość.

3.1 Równoległy Multi-Start (Parallel Multi-Start) (C++)

Wewnątrz modułu C++ zaimplementowano wielowątkowość korzystając z `<thread>` z biblioteki standardowej. Po uruchomieniu algorytmu automatycznie wykrywana jest dostępna liczba rdzeni logicznych procesora za pomocą `std::thread::hardware_concurrency()` lub pobierana jest wartość wprowadzona przez użytkownika i tworzona jest odpowiednia liczba niezależnych wątków.

Architektura algorytmu opiera się na modelu Równoległego Multi-Startu (*Parallel Multi-Start*). Każdy z wątków uruchamia niezależną instancję algorytmu Tabu Search (funkcja `tabu_search_watek`). Skuteczność zapewnia różnicowanie poszukiwań poprzez tworzenie dla każdego wątku własnego generatora liczb pseudolosowych (`std::mt19937`). Pierwsza wartość pseudolosowa (`seed`) przekazana jest na podstawie identyfikatora wątku (`1337 + id_watku`), w ten sposób każdy z wątków startuje z inną losowo zaburzoną macierzą startową i podąża inną ścieżką.

Wątki mają jedynie dostęp do wspólnego pola (`najlepsze_globalne`), w którym to przechowywane jest najlepsze dotychczasowe rozwiązanie. Aby zapobiec błędom spowodowanym przez jednoczesne nadpisywanie najlepszego rekordu, sekcja krytyczna zabezpieczona jest blokadą wzajemnego wykluczania przy użyciu `std::mutex`.

3.2 Protokół komunikacji z interfejsem

Aby interfejs graficzny napisany w Tkinter (Python) nie zatrzymywał się w trakcie długotrwałych (wielominutowych) obliczeń, proces C++ wywoływany jest asynchronicznie poprzez `subprocess.Popen`.

Komunikacja między procesami prowadzona jest jednokierunkowo z użyciem standardowego `stdout`. W kodzie C++ wymuszone zostało buforowanie liniowe za pomocą `setvbuf(stdout, nullptr, _IOLBF, 0)`, co pozwala na natychmiastowe przesyłanie logów do potoku. Wątek roboczy wysyła komunikaty w ściśle określonym formacie:

- **PROGRESS:** `[iteracja]` – wysyłany okresowo komunikat z aktualnym postępem, pozwalający interfejsowi na aktualizację paska postępu.
- **CHART:** `[iteracja], [wynik]` – zdarzenie wysyłane natychmiast, gdy któryś z wątków znajdzie nowe, lepsze rozwiązanie lokalne bez konfliktu.

Po stronie Pythona oczekiwanie na `stdout` z procesu C++ wykonywane jest w osobnym wątku daemon (`threading.Thread`). Parser używa wyrażeń regularnych do czytania przychodzących poleceń, a następnie korzystając z bezpiecznej dla wątków metody

`root.after()`, zleca aktualizację w interfejsie oraz rysowanie wykresu zbieżności przy użyciu `Matplotlib` do głównego wątku. Dzięki temu podejściu, program działa płynnie, a w interfejsie na bieżąco widzimy wizualizację postępu poszukiwań metaheurystyki.

4 Algorytm Tabu Search

Postawiony problem optymalizacyjny rozwiązany został poprzez zaimplementowaną metaheurystykę Tabu Search, bazującą na lokalnym poszukiwaniu z pamięcią krótkoterminową. Implementacja algorytmu została odpowiednio dostosowana do problemu dopasowania wielu sekwencji poprzez odpowiednią reprezentację rozwiązania oraz operatory sąsiedztwa dostosowane do danej reprezentacji.

4.1 Reprezentacja rozwiązania

Pojedyncze rozwiązanie (stan przestrzeni poszukiwań) jest tutaj reprezentowane przy pomocy struktury `Rozwiazanie`, która zawiera macierz dopasowania zapisaną jako wektor ciągów znaków (`std::vector<std::string>`). Każdy wiersz reprezentuje kolejną sekwencję, natomiast w komórkach znajdują się nukleotydy (A, C, G, T) lub luki (_). Wykorzystanie standardowych ciągów znaków pozwala na bardzo wydajne operacje wstawiania, usuwania oraz przesuwania fragmentów pamięci bez konieczności ponownej alokacji całej macierzy.

4.2 Funkcja celu (Dual-Objective Pipeline)

Biorąc pod uwagę sztywny warunek braku konfliktów w kolumnach, funkcja celu rozwiązania problemu (`oceniaj`) została zaimplementowana jako funkcja z karą. Zdefiniowana następująco:

$$F(x) = L(x) + WK \cdot K(x)$$

gdzie:

- $L(x)$ to efektywna długość macierzy wyrównania (rozmiar najdłuższego wiersza z pominięciem pustych znaków na jego końcu),
- $K(x)$ to sumaryczna liczba niedopasowań we wszystkich kolumnach, obliczana na podstawie reguły względnej większości (*Plurality Consensus*),
- WK to waga kary za konflikty.

Tak zdefiniowana funkcja pozwala algorytmowi oceniać nie tylko długość, ale i wystąpienie w macierzy konfliktów.

4.3 Generowanie sąsiedztwa

W każdej iteracji algorytm generuje i ocenia $k = 20$ sąsiadów bieżącego rozwiązania. Generowanie sąsiada polega na wykonaniu jednego z dwóch dedykowanych ruchów (z 50% prawdopodobieństwem dla każdego):

- **Przesunięcie bloku** (`generuj_sasiada_blok`):

Algorytm losuje wiersz i pozycję, a następnie znajduje ciągły blok nukleotydów otoczony spacjami. Następnie przesuwa on ten blok o jedną pozycję w lewo lub w prawo, symulując poziome przesuwanie dopasowania.

- **Modyfikacja luk (generuj_sasiada_gap):**

Operator, który może wstawić zupełnie nową kolumnę pustych znaków w losowym miejscu wyrównania lub usunąć kolumnę złożoną z samych luk.

Algorithm 1 Wątek roboczy metaheurystyki Tabu Search (Równoległy Multi-Start)

Require: Instancja wejściowa M , max iteracji I , rozmiar sąsiedztwa k , limit czasu T
Ensure: Zaktualizowanie globalnego najlepszego rozwiązania $NajlepszeGlobalne$

```

1: if  $M$  jest pusta then return
2: end if
3:  $S_{current}, S_{lokalne\_naj} \leftarrow \text{LosujStartRozciagnij}(M)$ ,  $\tau \leftarrow 15 + \text{rand}(10)$ 
4:  $Koszt_{lokalny} \leftarrow f(S_{current}, 2000)$ ,  $TabuList \leftarrow \emptyset$ ,  $LicznikBrakPoprawy \leftarrow 0$ 
5:  $S_{best} \leftarrow \text{Napraw}(S_{current})$ ,  $Dlugosc_{best} \leftarrow \text{Dlugosc}(S_{best})$ 
6: for  $iter \leftarrow 0$  to  $I - 1$  do
7:    $WK \leftarrow (LicznikBrakPoprawy \bmod 100) < 15 ? 1 : 2000$ 
8:   if  $WK$  uległo zmianie then  $Koszt_{lokalny} \leftarrow f(S_{lokalne\_naj}, WK)$ 
9:   end if
10:  if  $iter \bmod 100 == 0$  and  $\text{CzasObliczen}() \geq T$  then break
11:  end if
12:   $S_{next} \leftarrow \text{null}$ ,  $Koszt_{next} \leftarrow \infty$ 
13:  for  $s \leftarrow 1$  to  $k$  do
14:     $S_{cand}, Ruch \leftarrow \text{GenerujSasiada}(S_{current})$ 
15:    if  $Ruch$  nie w  $TabuList$  or  $f(S_{cand}, WK) < Koszt_{lokalny}$  then
16:      if  $f(S_{cand}, WK) < Koszt_{next}$  then
17:         $S_{next} \leftarrow S_{cand}$ ,  $Koszt_{next} \leftarrow f(S_{cand}, WK)$ ,  $Ruch_{wybrany} \leftarrow Ruch$ 
18:      end if
19:    end if
20:  end for
21:  if  $S_{next} \neq \text{null}$  then
22:     $S_{current} \leftarrow S_{next}$ 
23:    if  $Koszt_{next} < Koszt_{lokalny}$  then
24:       $Koszt_{lokalny} \leftarrow Koszt_{next}$ ,  $S_{lokalne\_naj} \leftarrow S_{current}$ ,  $LicznikBrakPoprawy \leftarrow 0$ 
25:      if  $WK > 1$  and  $\text{Dlugosc}(\text{Napraw}(S_{current})) < Dlugosc_{best}$  then
26:         $S_{best} \leftarrow \text{Napraw}(S_{current})$ ,  $Dlugosc_{best} \leftarrow \text{Dlugosc}(S_{best})$ 
27:      end if
28:    else
29:       $LicznikBrakPoprawy \leftarrow LicznikBrakPoprawy + 1$ 
30:    end if
31:     $TabuList.\text{Push\_Back}(\text{OdwrotnoscRuchu}(Ruch_{wybrany}))$ 
32:    if  $|TabuList| > \tau$  then  $TabuList.\text{Pop\_Front}()$ 
33:  end if
34: else
35:    $LicznikBrakPoprawy \leftarrow LicznikBrakPoprawy + 1$ 
36: end if
37: if  $LicznikBrakPoprawy > 500$  then
38:    $S_{current}, S_{lokalne\_naj} \leftarrow \text{LosujStartRozciagnij}(M)$ ,  $TabuList \leftarrow \emptyset$ 
39:    $LicznikBrakPoprawy \leftarrow 0$ ,  $Koszt_{lokalny} \leftarrow f(S_{lokalne\_naj}, WK)$ 
40: end if
41: end for
42: if  $Dlugosc_{best} == \infty$  then  $S_{best} \leftarrow \text{Napraw}(S_{current})$ ,  $Dlugosc_{best} \leftarrow \text{Dlugosc}(S_{best})$ 
43: end if
44: SekcjaKrytyczna(Lock_Mutex)
45: if  $Dlugosc_{best} < NajlepszeGlobalne.funkcja\_celu$  then
46:    $NajlepszeGlobalne.sekwencje \leftarrow S_{best}$ ,  $NajlepszeGlobalne.funkcja\_celu \leftarrow Dlugosc_{best}$ 
47: end if
48: SekcjaKrytyczna(Unlock_Mutex)
```

4.4 Lista Tabu i Kryterium Aspiracji

Lista Tabu zaimplementowana jest w oparciu o kolejkę dwustronną (`std::deque<Ruch>`). Przechowuje ona odwrotności wykonanych ruchów (funkcja `odwrotnosc_ruchu`), co skutecznie zapobiega natychmiastowemu cofaniu korzystnych zmian (np. po przesunięciu bloku w prawo, na listę trafia zakaz przesunięcia go w lewo).

Rozmiar listy, czyli *Tabu Tenure*, jest dynamiczny i losowany z przedziału od 15 do 24. Dodanie elementu losowości do rozmiaru pamięci zapobiega wpadaniu algorytmu w cykliczne, zdegenerowane pętle. Zastosowano także klasyczne kryterium aspiracji: jeśli zakazany ruch prowadzi do rozwiązania lepszego niż absolutnie najlepsze znalezione do tej pory rozwiązanie lokalne, tabu jest ignorowane i ruch zostaje wykonany.

4.5 Heurystyka Naprawcza (Procedura Repair)

Optymalizacja z użyciem funkcji kary może doprowadzić do wygenerowania macierzy o najkrótszej długości, jednak zawierającej niedopuszczalne konflikty kolumnowe. Z tego powodu na etapie aktualizacji globalnego minimum wywoływana jest metoda `napraw()`.

Algorytm ten iteracyjnie skanuje kolumny szukając konfliktów (przypadków, gdy występuje więcej niż jeden nukleotyd). Po napotkaniu konfliktu, heurystyka rozważa dwa sposoby rozwiązania: wstawienie spacji dokładnie po znaku powodującym konflikt (co przesuwając pozostałe znaki o jedno miejsce w prawo) lub wypchnięcie znaku powodującego konflikt w prawo o więcej pozycji (maksymalnie `MAX_PRZESUNIEC_W_PRAWO = 6`). Algorytm testuje każdą z możliwości, ale aby nie sprawdzać wpływu zmiany na całą macierz, wykonuje lokalny podgląd, patrząc tylko na 8 najbliższych kolumn w przód (`LOOKAHEAD_COLUMN = 8`). Na koniec procedura pozbywa się rzadko, ale pojawiających się całkowicie pustych kolumn, gwarantując zwrot bezkonfliktowego i zbitego rozwiązania.

Algorithm 2 Heurystyka Naprawcza (Lookahead Repair)

Require: Macierz S z potencjalnymi konfliktami (np. odmienne nukleotydy w tej samej kolumnie)

Ensure: Zoptymalizowana macierz S' ze zminimalizowaną liczbą konfliktów (najlepsza znaleziona)

```
1: if  $S$  jest pusta then return  $S$ 
2: end if
3:  $S' \leftarrow S$ ,  $MaxOchrona \leftarrow \max(50000, |S'| \cdot 5000)$ 
4: for  $krok \leftarrow 0$  to  $MaxOchrona - 1$  do
5:   if not ZnajdzPierwszyKonflikt( $S'$ ,  $col_{bad}$ ,  $row_{bad}$ ) then break
6:   if
7:     then  $S_{temp} \leftarrow Kopia(S')$ ,  $S_{temp}.WstawLuke(row_{bad}, col_{bad})$ 
8:      $C_{best} \leftarrow KosztOkna(S_{temp}, col_{bad}, 8)$ ,  $Tryb \leftarrow 0$            ▷ Opcja 1: Wstawienie luki
9:     for  $k \leftarrow 1$  to 6 do           ▷ Opcja 2: Przesunięcia w prawo
10:       $S_{temp} \leftarrow Kopia(S')$ 
11:      if not MozaDalejPrzesunac( $S_{temp}$ ,  $row_{bad}$ ,  $col_{bad}$ ,  $k$ ) then break
12:      if
13:        then  $S_{temp}.PrzesunZnakZastepujacLuke(row_{bad}, col_{bad}, k)$ 
14:         $Koszt \leftarrow KosztOkna(S_{temp}, col_{bad}, 8)$ 
15:        ▷ W przypadku remisu kosztów preferowane jest przesunięcie
16:        if  $Koszt < C_{best}$  or ( $Koszt == C_{best}$  and  $Tryb == 0$ ) then
17:           $C_{best} \leftarrow Koszt$ ,  $Tryb \leftarrow 1$ ,  $IleSwap \leftarrow k$ 
18:        end if
19:      if  $Tryb == 0$  then  $S'.WstawLuke(row_{bad}, col_{bad})$ 
20:      else  $S'.PrzesunZnakZastepujacLuke(row_{bad}, col_{bad}, IleSwap)$ 
21:      end if
22:
23:
24:    $S' \leftarrow UsunCalkowiciePusteKolumny(S')$            ▷ Kompaktowanie rozmiaru
25: return  $S'$ 
```

4.6 Zaawansowane Mechanizmy Dywersyfikacji

Silna NP-trudność problemu uwidacznia się na dużych instancjach, na których algorytm szybko wpada w głębokie minima lokalne. W celu przeciwdziałania temu procesowi, zrealizowane zostały dwa mechanizmy:

4.6.1 Strategic Oscillation (Heartbeat)

Mechanizm realizujący koncepcję oscylacji poprzez pulsacyjne zmiany wagi kary (WK) w funkcji celu. Jej domyślna wartość została ustalona na stałe na 2000, co zmusza algorytm do poszukiwania rozwiązań wolnych od konfliktów. Jednak jeżeli po jakimś czasie staną się one niedostępne, co 100 iteracji przez dokładnie 15 kroków waga kary obniżona zostaje do 1. Takie podejście pozwala na chwilowe ignorowanie konfliktów, żeby wydostać się z głębokiego minimum lokalnego. Po dokładnie 15 iteracjach waga kary powraca do wartości 2000, co ponownie zmusza algorytm do uporządkowania macierzy z powstałych konfliktów.

4.6.2 Heavy Diversification (Kick-Move)

Mechanizm typu *multi-start* włączany przy zjawisku utknięcia algorytmu w głębokim minimum lokalnym. Jeżeli przez 500 iteracji nie uda się przebić najlepszego możliwego lokalnego wyniku, to oznacza, że dany obszar został już przez algorytm wyczerpany. W tym momencie następuje dywersyfikacja: wywołanie funkcji `losuj_start()` (powoduje rozciągnięcie macierzy, wprowadzając duże, losowo ułożone bloki luk), która opróżnia pamięć Tabu oraz zeruje liczniki. Wszystko dzieje się po to, aby przenieść poszukiwanie do zupełnie innego miejsca w przestrzeni.

5 Złożoność Obliczeniowa i Pamięciowa

Analiza złożoności obliczeniowej algorytmu Tabu Search do rozwiązywanego problemu *MIN LENGTH ALIGNMENT* jest konieczna, aby oszacować jego skalowalność wraz ze wzrostem rozmiarów danych wejściowych.

W celu oceny złożoności przyjęto następujące definicje parametrów:

- m – Liczba sekwencji (wierszy).
- L – Aktualna długość dopasowania (liczba kolumn).
- k – Rozmiar badanego sąsiedztwa w pojedynczej iteracji.
- I – Całkowita liczba iteracji.
- T – Liczba uruchomionych wątków.

5.1 Złożoność czasowa funkcji celu

Najczęściej wywoływaną funkcją w ramach algorytmu jest funkcja oceny (`oceniaj`), która jest wywoływana dla każdego wygenerowanego sąsiada. Złożoność wyliczenia funkcji celu zależy wprost od wielkości macierzy dopasowania. Funkcja `rozklad_oceny` przeszukuje wszystkie m sekwencji, aby znaleźć efektywną długość ($O(m \cdot L)$), a następnie wywołuje algorytm `suma_niedopasowan_do_plurality`. Algorytm ten przeszukuje kolejno kolumny, licząc wystąpienia nukleotydów (A, C, G, T) w m wierszach. Sumaryczny koszt jednej pełnej ewaluacji wynosi zatem $O(m \cdot L)$.

5.2 Złożoność generowania sąsiedztwa i pętli głównej

W każdej iteracji algorytm losowo generuje i ocenia k sąsiadów (gdy $k = 20$). Generowanie sąsiada (np. `generuj_sasiada_blok` lub `generuj_sasiada_gap`) wymaga skopionowania obecnego dopasowania do nowej lokalnej zmiennej (koszt $O(m \cdot L)$) oraz wykonania wstawienia/przesunięcia w czasie optymistycznym $O(L)$ (z użyciem `std::rotate` lub `std::vector::insert`).

Złożoność pojedynczej iteracji wynosi w przybliżeniu $O(k \cdot m \cdot L)$. Natomiast teoretyczna złożoność czasowa uruchomienia metaheurystyki dla jednego wątku wynosi zatem $O(I \cdot k \cdot m \cdot L)$ (wielomianowa złożoność), co zapewnia jej wysoką responsywność i przewidywalność czasu działania, rosnącą liniowo względem gęstości problemu.

5.3 Złożoność procedury naprawczej

Dostosowana do problemu heurystyka `napraw()` jest wywoływana tylko wtedy, gdy znaleziono nowe najlepsze rozwiązanie lokalne. W najgorszym przypadku przesuwają się one po wszystkich kolumnach i dla każdego konfliktu sprawdza lokalne okno za pomocą heurystyki *lookahead* o promieniu (domyślnie 8 kolumn). Choć operacje te wiążą się z lokalnym narzutem iteracyjnym, z uwagi na ich rzadkość wywoływania, w ujęciu uśrednionym procedura ta niemal nie wpływa na ostateczną złożoność czasową.

5.4 Narzut pamięciowy (Space Complexity)

Opisana implementacja cechuje się niezwykle niskim narzutem na pamięć. Całkowita złożoność pamięciowa dla jednego wątku to jedynie $\mathcal{O}(m \cdot L)$. Główne czynniki wpływające na zużycie pamięci to:

- **Macierze robocze:** W danym momencie w ramach pojedynczego wątku potrzebne są jedynie niezbędne instancje wektora ciągów znakowych: macierz instancji, bieżące rozwiązanie, najlepsze rozwiązanie lokalne, globalne z wątku oraz zmienna pomocnicza dla rozpatrywanego sąsiada.
- **Lista Tabu:** Zamiast kosztownego pamięciowo zapisywania pełnych odwiedzonych macierzy (co bardzo szybko doprowadziłoby do wyczerpania dostępnej pamięci RAM dla większych instancji), kolejka `std::deque` przechowuje jedynie odwracalne operacje. Struktura `Ruch` zajmuje kilkanaście bajtów (typ wbudowany jako 8-bitowy *enum* oraz cztery liczby całkowite definiujące: wiersz, kolumnę początkową oraz kierunek). Nawet w przypadku maksymalnego ustawienia *Tabu Tenure*, złożoność pamięciowa listy Tabu jest rzędu $\mathcal{O}(1)$.

Zrealizowana wielowątkowość bazująca na *Równoległy Multi-Start* powiela wspomniany, drobny koszt $\mathcal{O}(m \cdot L)$ (dla każdego uruchomionego wątku $\mathcal{O}(T \cdot m \cdot L)$), co jest kosztem niemal pomijalnym.

6 Interfejs użytkownika (UI) i Generator Instancji

W ramach zadania zaprojektowana i zaimplementowana została osobna aplikacja z graficznym interfejsem użytkownika (GUI) napisana w Python z użyciem biblioteki `Tkinter`. Aplikacja ta pełni trzy role: generatora instancji z możliwością wprowadzenia swoich parametrów, panelu sterowania metaheurystyką oraz dynamicznego widoku zbieżności.

6.1 Generator instancji i wprowadzanie szumu

W pierwszym oknie interfejsu, widocznym na samym początku kompilacji aplikacji, możliwe jest generowanie nietrywialnych instancji problemu MSA w sposób kontrolowany. Użytkownik definiuje podstawowe parametry rozmiaru:

- Liczbę sekwencji (m),
- Długość pojedynczej sekwencji (n),
- Długość nadciągu bazowego (d), z którego losowo wycinane są sekwencje podrzędne.

Zaimplementowana została walidacja danych wejściowych wpisywanych przez użytkownika, która weryfikuje poprawność typów danych oraz wymusza logiczny warunek $n \leq d$ (podciąg nie może być dłuższy niż nadciąg z którego pochodzi).

Dodatkowo zaimplementowano funkcje kluczowe z perspektywy eksperymentalnej:

- **Wybór alfabetu:** Opcja przełączania między pełnym alfabetem DNA (ACGT) a ułatwionym alfabetem binarnym (AC).
- **Wprowadzanie mutacji:** Mechanizm wprowadzający kontrolowany szum. Funkcja podmienia losowe znaki w wygenerowanych sekwencjach na inne, dozwolone litery wybranego alfabetu, co pozwala na testowanie odporności algorytmu na zaburzenia struktury wejściowej.
- **Edycja manualna i Pusty wzorzec:** Umożliwia ręczne wpisywanie lub modyfikowanie wygenerowanych instancji bezpośrednio w wbudowanym polu tekstowym.

6.2 Konfiguracja i parametryzacja procesów obliczeniowych C++

W oddzielnej sekcji GUI, użytkownik może ustawić kluczowe parametry działającej metaheurystyki przed jej uruchomieniem: limit czasowego działania (w sekundach), maksymalną liczbę iteracji oraz ilość wątków (domyślnie wartość liczbę fizycznych rdzeni `os.cpu_count()`). Po naciśnięciu przycisku `Uruchom obliczenia` Python przed zapisywaniem do pliku bezwzględnie tasuje wiersze instancji (`random.shuffle()`), zacierając ślady po pierwotnym, ułożonym nadciągu, co stanowi podstawowy wymóg poprawności.

6.3 Wizualizacja zbieżności i raportowanie

Dla przejrzystości stworzone zostało oddzielne okno wyników, które w pełni ukazuje pracę modułu obliczeniowego w czasie rzeczywistym. Do wizualizacji wykorzystano bibliotekę `Matplotlib` osadzoną w `Tkinterze` (`FigureCanvasTkAgg`).

Na podstawie asynchronicznie parsowanych logów z procesu `C++` (komunikaty `PROGRESS` oraz `CHART`), aplikacja na bieżąco rysuje krzywą zbieżności funkcji celu za pomocą wykresu schodkowego (`drawstyle="steps-post"`). Wykres dynamicznie dopasowuje granice osi (X oraz Y), aby zawsze centralnie obrazować spadek długości wyrównania w czasie. Użytkownik widzi również postęp wyrażony poprzez wbudowane paski postępu (`ttk.Progressbar`) dla czasu oraz iteracji.

Po zakończeniu obliczeń aplikacja parsowała zwrócony plik wynikowy w celu wydobycia dokładnych statystyk (długość, czas `C++`, liczba konfliktów) i wyświetlenia ostatecznej macierzy z sekwencją konsensusową. Zaimplementowano również opcję "Zapisz raport", pozwalającą wyeksportować wszystkie użyte parametry generatora, ustawienia metaheurystyki oraz logi wyjściowe do pliku tekstowego.

Problem VII - generator instancji

Generator instancji

Liczba sekwencji m:

Długość sekwencji n:

Długość nadciągu d:

Alfabet:

Wygeneruj czystą instancję

Pusty wzorzec (Wyczyść)

Liczba mutacji:

Dodaj mutacje

Zrozumienie optymalności: Znana minimalna długość ≤ 60 (czysta instancja)

Sekwencje wejściowe (możesz edytować ręcznie):

```

CAACCCCAATGTCACAAGTTCCACGGGCGCTGGTCACCTTTAAGCGAGTAA
CAACCCCTCAATGTCACAAAGTCCGCGGGCCTGGTCTACTAAGGTGATAAC
CAACTCATGCAACAAAGTTCCGAGGGCGCTGATCCTAAGCGTAGTATAAC
CAACCCCTCAATGTCACAAAGTTCGACGGGCGTGATCTCTAAGCGTAAAC
AACCTATGCAACAAGTTCGACGGGCGCTGATCTACTTTAAGCTGTATAAC
CAACCCCTCATGTCACAAGTCCGACGGGCGTGTTCTTTAAGGTAGTATAC
AACCTCAATGTCAACAGTCCGACGGGCTGAGTCTACTTTAAGCGTAGTATA
CAACCCCTCATGCAACAAAGTTCGAGGGCGCGTCTACTTTAAGCGTAGTATA
CACCTCAATGTCACAAAGTTCGACGGGCTGAGTCTCTTTAGCGTAGTAAAC
CACCCATGTCACAAAGTTCGACGGGCGTAGTCTACTTTAAGCGTAGTAAAC

```

Tabu Search (silnik C++)

Liczba wątków:

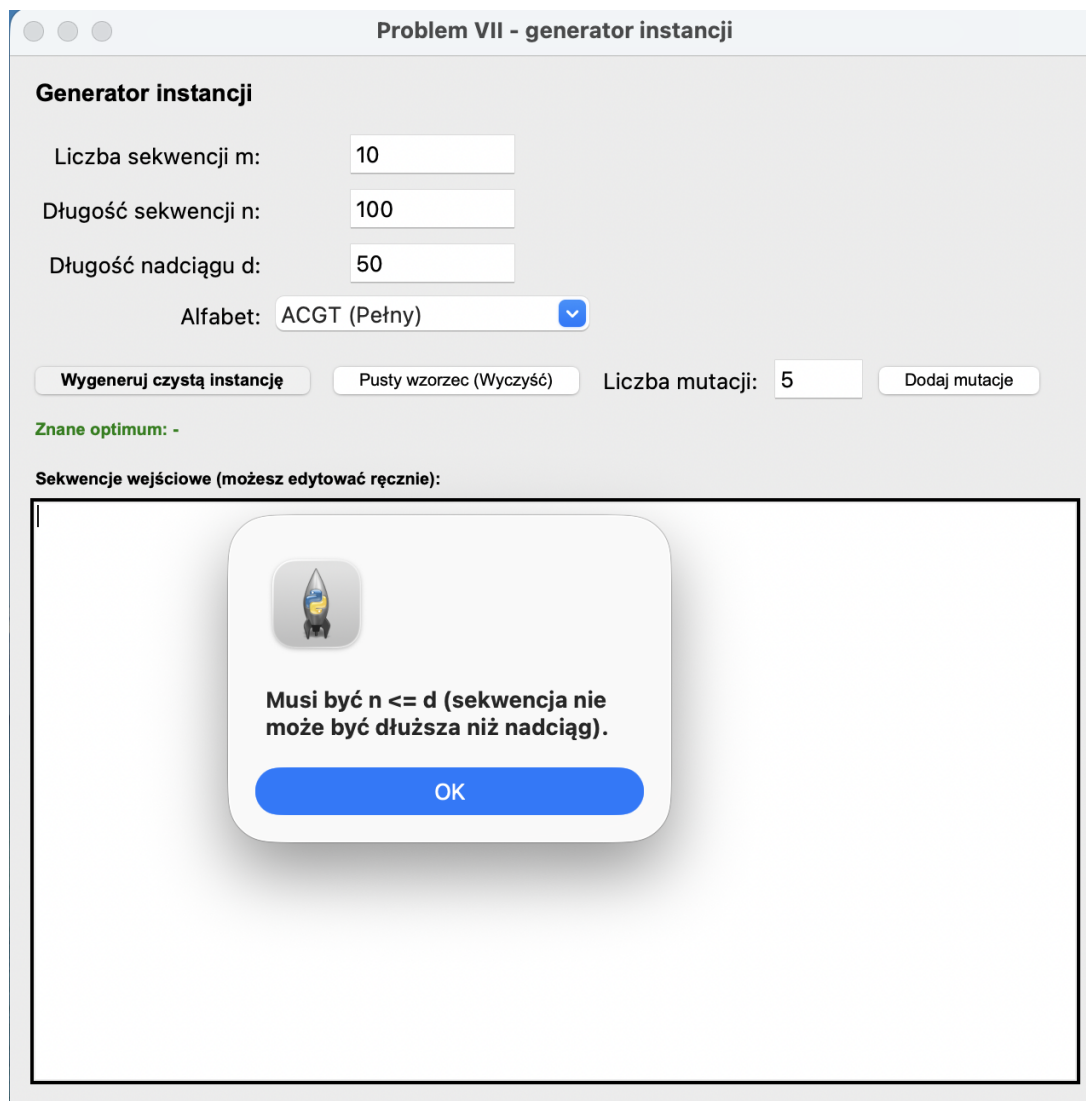
Limit czasu [s]:

Limit iteracji:

Uruchom obliczenia

Pokaż okno wyników

Rysunek 1: Główne okno generatora instancji problemu MSA oraz sekcji parametryzacji wielowątkowego modułu obliczeniowego C++.



Rysunek 2: Komunikat ostrzegawczy systemowego okna dialogowego (MessageBox) w przypadku wykrycia naruszenia warunku walidacji danych wejściowych ($n > d$).

Problem VII - generator instancji

Generator instancji

Liczba sekwencji m:

Długość sekwencji n:

Długość nadciagu d:

Alfabet: 

Liczba mutacji:

Zrozumienie optymalności: Znana minimalna długość ≤ 60 (czysta instancja)

Sekwencje wejściowe (możesz edytować ręcznie):

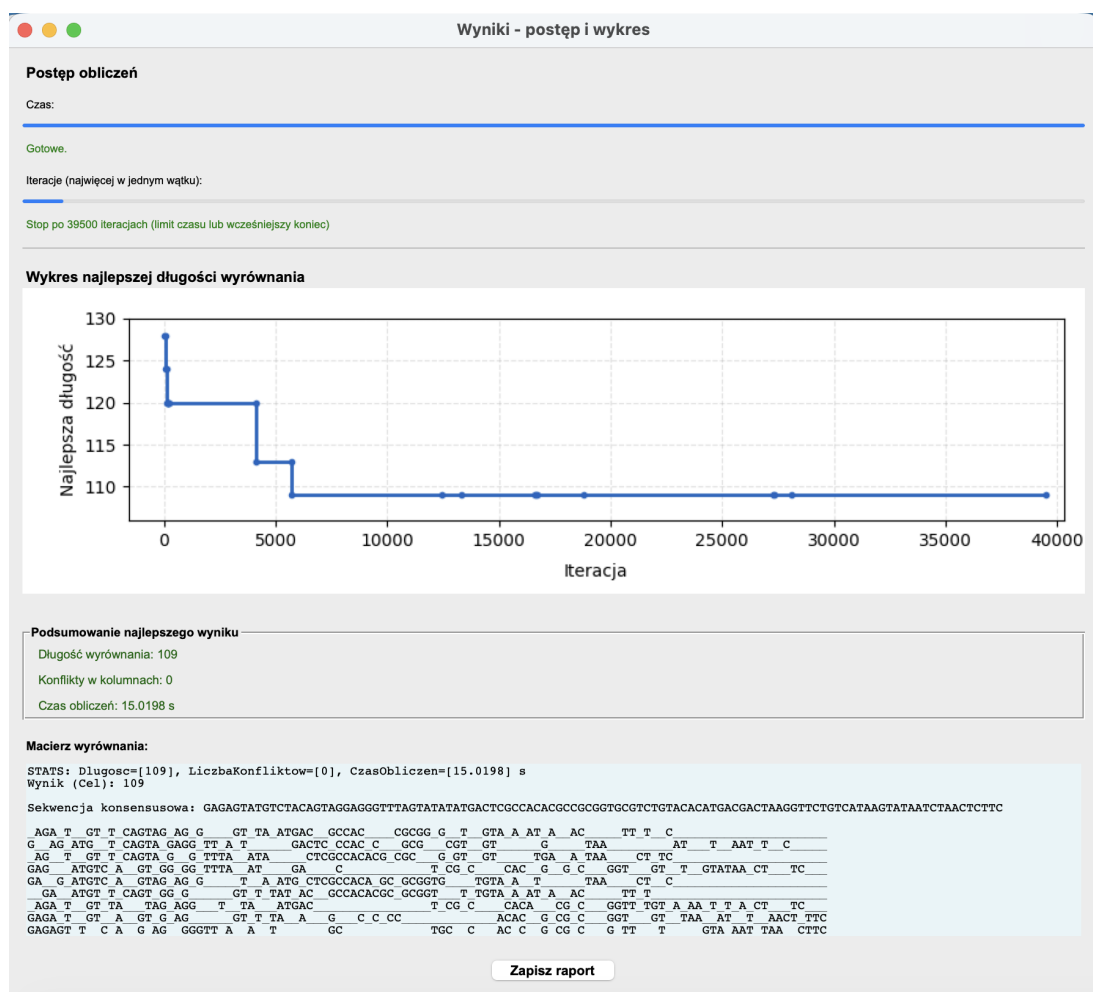
```

AGTGCAGTCCCATGCAAGTAA
AGGTGCAGTCCCATCAAGGT
AGGTGCAGTCCCTGCAAGTT
AGGTGCAGTCCAGCAAGGTT
AGGTGCAGCCTGCAAGGTTA
GGTGCAGTCCATCAAGGTTA
AGGTGCAGTCCAGCAAGGTT
AGGGAGTCCATGAAGGTTAA
AGTGCAGTCCATGCAAGGTT
AGTGCAGCCCAGCAAGTTT
  
```



Podaj poprawną liczbę mutacji.

Rysunek 3: Działanie mechanizmu walidacji typów danych przy próbie wprowadzenia niepoprawnych znaków niebędących liczbami całkowitymi.



Rysunek 4: Okno wizualizacji zbieżności w czasie rzeczywistym, prezentujące paski postępu obliczeń oraz schodkową krzywą optymalizacji funkcji celu.

```

=====
RAPORT – Problem VII (Tabu Search + UI Python)
=====
Data zapisu raportu: 2026-06-08T20:51:11
Start obliczen: 2026-06-08T20:49:16
Koniec obliczen: 2026-06-08T20:49:31

--- Parametry generatora ---
Liczba sekwencji m: 10
Dlugosc sekwencji n: 50
Dlugosc nadciagu d: 60
Alfabet: ACGT (Pełny)
Liczba dodanych mutacji: 0

--- Parametry silnika C++ ---
Program: /Users/oskar/CLionProjects/zaawansp/bg/tabu_search
Watki: 8
Limit czasu [s]: 15
Limit iteracji: 1000000

--- Instancja wejsciowa (instancja.txt) ---
AGATGTTTCAGTAGAGGGTTAATGACGCCACCGCGGTGTAAATAACTTTC
GAGATGTCAGTAGAGGTTATGACTCCACCGCGGTGTAAATAACTTC
AGTGTTCAGTAGGTTTAACTACGCCACACGCGGTGTAAATAACTTC
GAGATGTCAGTAGGGGTTTAACTACGCCACGCGGTGTAAATAACTTC
GAGATGTCAGTAGAGGTAATGCTCGCCACACGCGGTGTAAATAACTTC
GAATGTTTCAGTAGGGGTTTATACGCCACACGCGGTGTAAATAACTTC
AGATGTTATAGAGGTTAATGACTCGCCACACGCGGTGTAAATAACTTC
GAGATGATGAGGTTTAAAGCCACACGCGGTGTAAATAACTTC
GAGAGTTCAGAGGGTTAATGCTGCCACCGCGGTGTAAATAACTTC
GAGAGTTCGTGGGTTAATACTCGCCACACGCGGTGTAAATAACTTC

--- Plik wynik.txt (ostatnie uruchomienie) ---
STATS: Dlugosc=[109], LiczbaKonfliktow=[0], CzasObliczen=[15.0198] s
Wynik (Cel): 109

Sekwencja konsensusowa: GAGAGTATGTCTACAGTAGGAGGGTTAGTATATATGACTCGCCACACCGCGGTGCGTCTGTACACATGACGACTAAGGTTCTGTACATAAGTATAATCTAACTTTC

_AGA_T_GT_T_CAGTAG_AG_G__GT_TA_ATGAC_GCCAC__CGCGG_G_T__GTA_A_AT_A__AC__TT_T_C
G_AG_ATG_T_CAGTA_GAGG_TT_A_T__GACTC_CCAC_C__GCG__CGT__GT__G__TAA__AT__T__AAT_T_C
_AG_T_GT_T_CAGTA_G_G_TTTA__ATA__CTCGCCACACG_CGC__G_GT__GT__TGA__A_TAA__CT_TC
GAG__ATGTC_A__GT_GG_GG_TTTA__AT__GA__C__T_CG_C__CAC__G_G_C__GGT__GT__T__GTATAA_CT__TC
GA__G_ATGTC_A__GTAG_AG_G__T_A_ATG_CTCGCCACA_GC_GCGGTG__TGTA_A_T__TAA__CT_C
__GA__ATGT_T_CAGT_GG_G__GT_T_TAT_AC_GCCACACGC_GCGGT__T_TGTA_A_AT_A__AC__TT_T
_AGA_T_GT_TA__TAG_AGG__T__TA__ATGAC__T_CG_C__CACA__CG_C__GGTT_TGT_A_AA_T_T_A_CT__TC
GAGA_T_GT__A__GT_G_AG__GT_TA_A__G__C_C_CC__ACAC__G_CG_C__GGT__GT__TAA__AT__T__AACT_TTC
GAGAGT_T_C_A__G_AG_GGGTT_A_A_T__GC__TGC__C__AC_C_G_CG_C__G_TT__T__GTA_AAT_TAA__CTTC
GAGAG__T_TC__GT__G_GG_TT_A__ATA__CTCGCCACACG_GCGGT__T_TGTA_A_AT__TAA__CT_TC
=====

```

Rysunek 5: Zrzut ekranu struktury wyeksportowanego raportu końcowego z badawczego logu aplikacji, widoczny w systemowym edytorze Notatnik.

7 Testy i Wyniki Eksperymentalne

W celu zweryfikowania skuteczności, stabilności i precyzji zaproponowanej wersji metaheurystyki Tabu Search, przeprowadzony został rozbudowany zbiór testów eksperymentalnych. Testy wykonano na komputerze osobistym z procesorem wielordzeniowym (co pozwoliło na test modelu *Równoległego Multi-Startu*) oraz odpowiednim zapasem pamięci RAM.

7.1 Metodologia badań i definicja miar

Zgodnie z zaleceniami zawartymi w poleceniu, w związku z losowym charakterem algorytmu, dla każdej konfiguracji parametrów utworzono **10 niezależnych** (z różnymi ziarnami (ang. *seed*) generatora) **instancji problemu**. Wyniki przedstawione w tabelach poniżej dotyczą minimalnych, maksymalnych i średnich wartości z tych 10 uruchomień.

Przyjęto następujące miary i parametry opisujące przebieg testów:

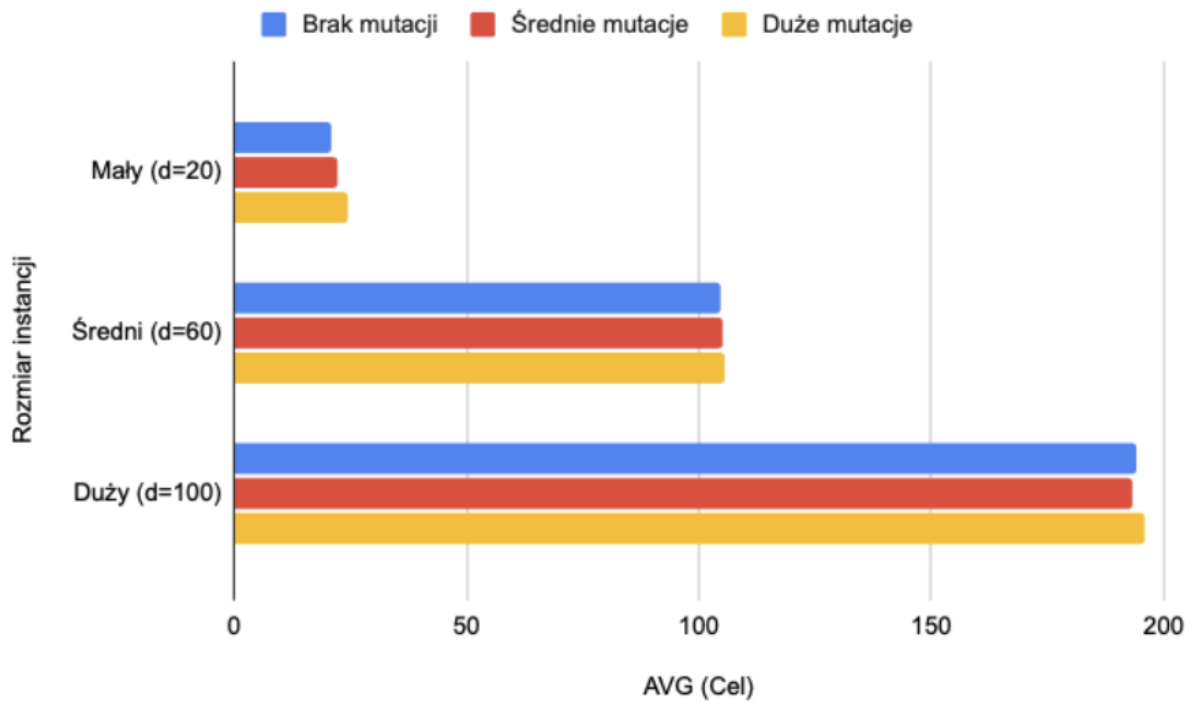
- m, n, d – parametry generatora (liczba sekwencji, długość sekwencji, długość nadciagu). Wartość d traktowana jest jako **zaszczepione optimum** ($\leq d$) dla czystych instancji.
- **Mutacje** – liczba celowo wprowadzonych błędów (substytucji) zacierających pierwotne optimum.
- **Liczba konfliktów** – miara weryfikująca poprawność procedury `napraw()`. Zgodnie z definicją problemu, w rozwiązaniu końcowym wartość ta musi zawsze wynosić 0.
- **AVG, MIN, MAX** – odpowiednio: średnia, najmniejsza (najlepsza) i największa (najgorsza) całkowita długość macierzy dopasowania ze wszystkich 10 uruchomień.
- **Gap (%)** – odległość uśrednionego wyniku (AVG) od znanego optimum (d), wyrażona w procentach. Obliczana ze wzoru: $Gap = \frac{AVG-d}{d} \cdot 100\%$.
- **Śr. Iteracji** – średnia liczba iteracji wykonanych przez najszybszy wątek w zadanym czasie.
- **Czas [s]** – limit czasu lub rzeczywisty czas pracy modułu C++.

7.2 Skalowalność i analiza odporności na szum (Mutacje)

Celem takiego eksperymentu jest sprawdzenie, w którym momencie NP-trudna natura problemu zacznie przeważać nad możliwościami eksploracyjnymi metaheurystyki. Ponadto, zgodnie z poleceniem, **dla każdego rozmiaru** dodano różne udziały błędów (mutacji), aby zweryfikować odporność algorytmu na szum. Czas wykonania pojedynczej próby ograniczono do 15 sekund, przy $k = 20$ i $T = 8$.

Rozmiar	m / n / d	Mutacje	Czas [s]	Konflikty	Śr. Iteracji	MIN	MAX	AVG (Cel)	Gap (%)
3*Mały	5 / 15 / 20	0	15	0	444654	20	22	21.1	5.5%
	5 / 15 / 20	2	15	0	461594	21	24	22.4	12.0%
	5 / 15 / 20	5	15	0	429100	22	26	24.6	23.0%
3*Średni	10 / 50 / 60	0	15	0	87450	100	112	104.7	74.5%
	10 / 50 / 60	5	15	0	93350	96	111	105.2	75.3%
	10 / 50 / 60	15	15	0	89113	97	114	105.6	76.0%
3*Duży	15 / 80 / 100	0	15	0	25000	188	208	194.2	94.2%
	15 / 80 / 100	10	15	0	24650	181	207	193.0	93.0%
	15 / 80 / 100	30	15	0	23500	189	203	195.9	95.9%

Tabela 1: Wpływ rozmiaru instancji oraz mutacji na jakość zbieżności algorytmu.



Rysunek 6: Zestawienie średniej długości wyrównania dla różnych rozmiarów instancji i poziomu szumu.

Wnioski z eksperymentu:

Jak możemy zobaczyć w Tabeli 1, dla mniejszych instancji ($m = 5$) metaheurystyka bez większych trudności kompresuje matrycę do okolic globalnego optymalnego rozwiązania (Gap wynosi jedynie 5.5% dla instancji bez szumu). Dodatkowo, heurystyka naprawcza w 100% przypadków nie zwraca żadnych konfliktów, zapewniając tym samym całkowitą poprawność biologiczną otrzymanych rozwiązań.

Wraz z wzrostem problemu (większe instancje) różnica do rozwiązania teoretycznie optymalnego znacząco się zwiększa, co jest dość standardowe dla NP-trudnych problemów.

Wprowadzenie mutacji zmusza algorytm do wstawiania dodatkowych luk buforujących, co naturalnie i w sposób kontrolowany zwiększa końcową wartość funkcji celu w zależności od stopnia zaszumienia.

7.3 Optymalizacja parametrów metaheurystyki

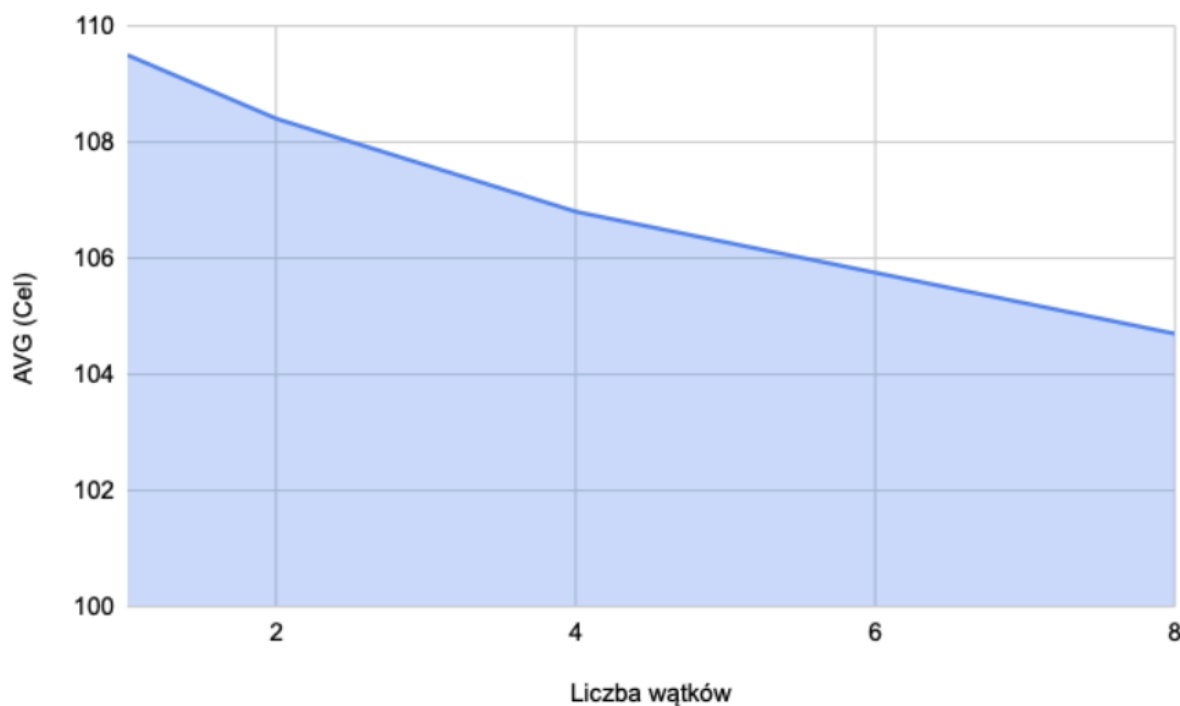
Kolejnym etapem badań było sprawdzenie, na ile poszczególne parametry sterujące działaniem algorytmu Tabu Search wpływają na końcowy efekt pracy. Pozwala to na udowodnienie, iż wartości zastosowane w implementacji nie są przypadkowe, lecz są wynikiem przemyślanych wyborów. Testy wykonano na ustalonej instancji problemu średniej wielkości ($m = 10, n = 50, d = 60$, 5 mutacji, czas 15 s).

7.3.1 Wpływ dywersyfikacji wielowątkowej (Liczba wątków T)

Eksperyment weryfikujący, w jakim stopniu zrównoleglenie obliczeń (przeszukiwanie przestrzeni z różnymi ziarnami (ang. *seed*) losowości) wpływa na unikanie minimów lokalnych. Przy stałym rozmiarze sąsiedztwa $k = 20$, zmieniano liczbę uruchomionych wątków roboczych.

Wątki (T)	Konflikty	MIN	MAX	AVG (Cel)	Gap (%)
1 wątek	0	99	118	109.5	82.5%
2 wątki	0	98	117	108.4	80.7%
4 wątki	0	98	114	106.8	78.0%
8 wątków (domyślnie)	0	98	114	104.7	74.5%

Tabela 2: Skuteczność architektury Równoległego Multi-Startu w zależności od liczby wątków.



Rysunek 7: Spadek średniej wartości funkcji celu w miarę dodawania niezależnych wątków roboczych.

Wnioski z eksperymentu:

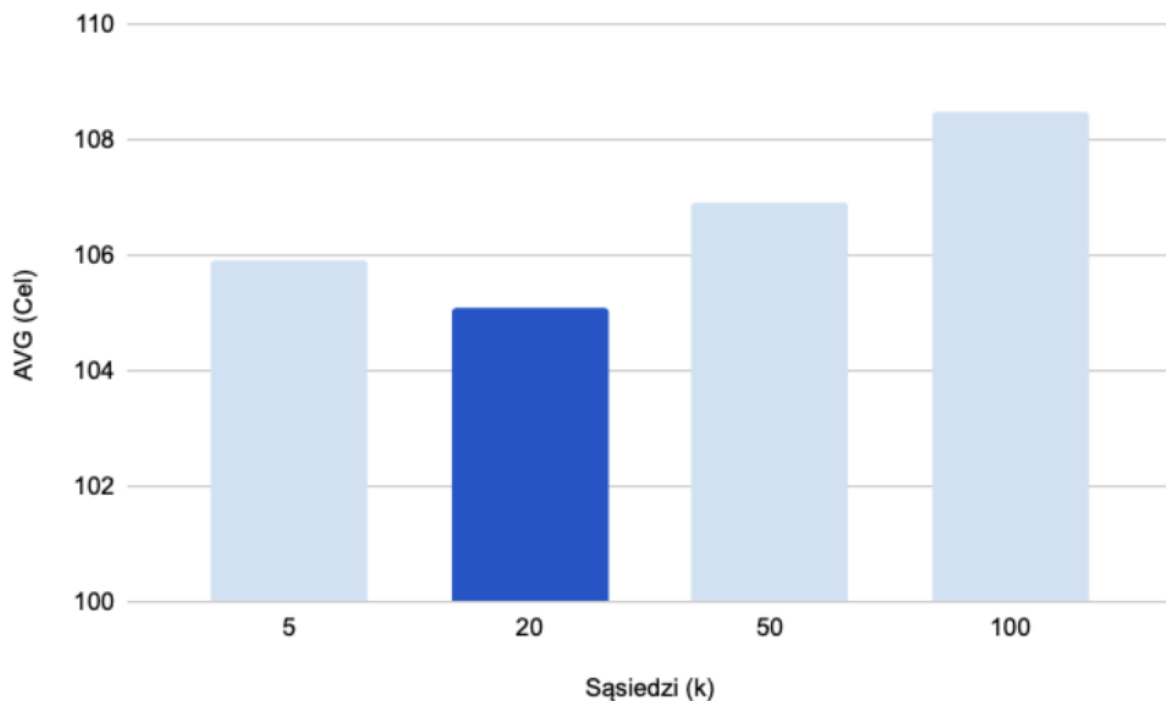
Zastosowanie architektury wielowątkowej okazało się kluczowe. Uruchomienie algorytmu w trybie pojedynczego wątku ($T = 1$) niesie ze sobą ryzyko utknięcia w głębokim optimum lokalnym (co pokazuje wysoka wartość MAX i AVG). Dodanie kolejnych wątków, z których każdy przemierza inną ścieżkę przestrzeni, znacząco obniża zakres wyników i poprawia średnią jakość dopasowania, przy jednoczesnym braku zmian w czasie wykonywania aplikacji (testy zawsze były przeprowadzane przy limicie 15 sekund).

7.3.2 Rozmiar generowanego sąsiedztwa (k)

Rozmiar sąsiedztwa to podstawowy parametr określający kompromis między eksploatacją (dokładnym sprawdzaniem okolic) a eksploracją (szybkim przemieszczaniem się w głąb przestrzeni). Testy przeprowadzono na 8 wątkach.

Sąsiedzi (k)	Konflikty	Śr. Iteracji	MIN	MAX	AVG (Cel)	Gap (%)
5 sąsiadów	0	281600	99	111	105.9	76.5%
20 (domyślne)	0	101550	100	109	105.1	75.2%
50 sąsiadów	0	46000	99	117	106.9	78.2%
100 sąsiadów	0	24150	103	118	108.5	80.8%

Tabela 3: Wpływ rozmiaru badanego sąsiedztwa na liczbę wykonanych iteracji i zbieżność.



Rysunek 8: Zależność uśrednionej wartości funkcji celu od rozmiaru generowanego sąsiedztwa (k).

Wnioski z eksperymentu:

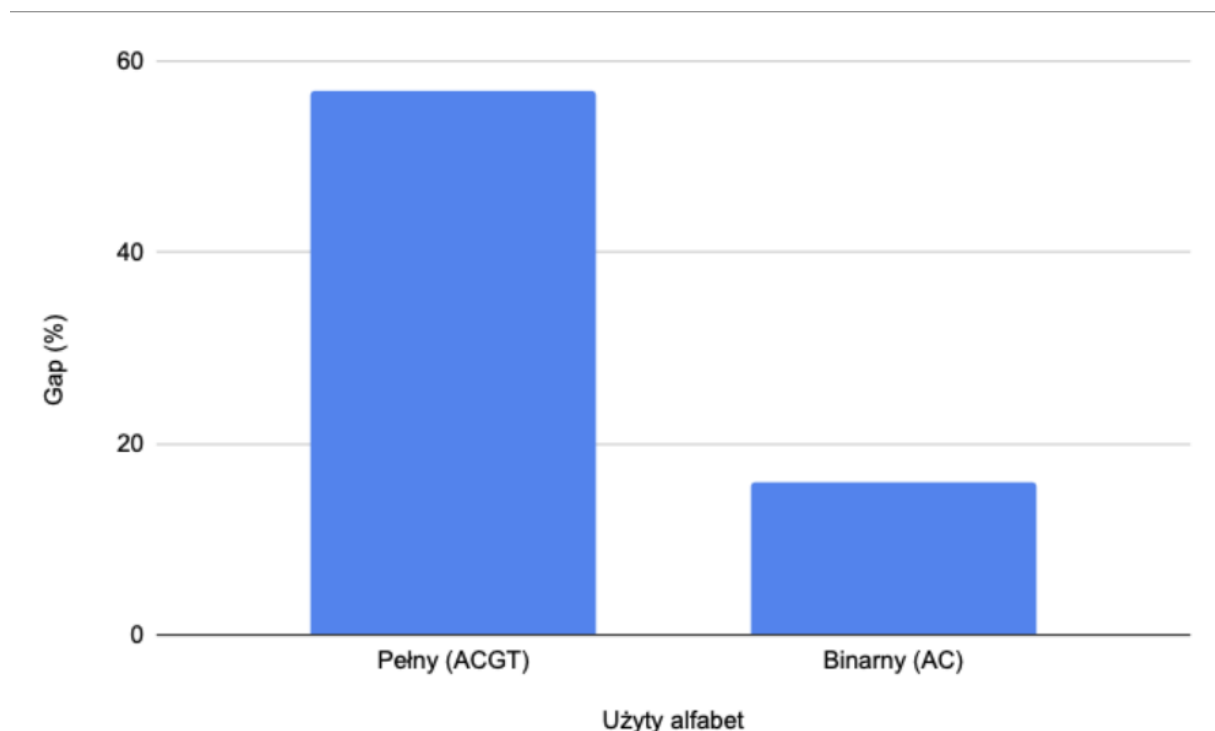
Za małe sąsiedztwo ($k = 5$) powoduje, że algorytm przechodzi bardzo powolnie przez przestrzeń rozwiązań i w bardzo wielu przypadkach akceptuje gorsze sąsiednie rozwiązania, co wydłuża czas dojścia do optymalnego rozwiązania. Natomiast zbyt duże sąsiedztwo ($k = 100$) ze względu na wysokie obciążenie obliczeniowe przeprowadzaną oceną funkcji celu $\mathcal{O}(m \cdot L)$, ogranicza liczbę wykonanych iteracji w ciągu 15 sekund do bardzo niskiej wartości, przez co algorytm nie ma szansy, by przejść dalej, przejrzeć dokładniej znaczącą przestrzeń. Wartość $k = 20$ to bardzo dobry kompromis.

7.4 Złożoność problemu a rozmiar użytego alfabetu

Ostatni test weryfikuje wpływ trudności instancji na czas działania. Porównano wyniki dla pełnego alfabetu DNA (ACGT) oraz sztucznie zredukowanego alfabetu (AC). Testy wykonano dla instancji $m = 10, n = 30, d = 40$ (bez mutacji, czas 10 sekund).

Użyty alfabet	Konflikty	MIN	MAX	AVG (Cel)	Gap (%)
Pełny {A, C, G, T}	0	59	67	62.7	56.8%
Binarny {A, C}	0	44	48	46.4	16.0%

Tabela 4: Różnice w kompresji macierzy w zależności od stopnia różnorodności alfabetu.



Rysunek 9: Porównanie długości optymalnego wyrównania dla różnych alfabetów.

Wnioski z eksperymentu:

Ograniczenie alfabetu znacząco zwiększa prawdopodobieństwo naturalnego ułożenia się znaków w bezkonfliktowe kolumny. Przy alfabecie dwuznakowym, przestrzeń rozwiązań

staje się prostsza, a heurystyka naprawcza prawie w ogóle nie musi wstawiać luk, co pozwala algorytmowi wygenerować nieporównywalnie lepsze wyrównanie (Gap bliski 0).

8 Wnioski i kierunki dalszych prac

Rezultaty eksperymentów badawczych pokazują, że opracowany wariant wielowątkowy metaheurystyki Tabu Search jest bardzo skutecznym narzędziem do rozwiązywania ściśle ograniczonego wariantu problemu *MIN LENGTH ALIGNMENT*. Budowa hybrydowej aplikacji wykorzystującej moc obliczeniową modułu C++ oraz elastyczność i możliwości analityczne interfejsu w języku Python sprostały oczekiwaniom stawianym podczas etapów projektowych.

8.1 Analiza wydajności i stabilności

Na podstawie przeprowadzonych eksperymentów na powstałym algorytmie można sformułować następujące wnioski:

Zalety zaproponowanej metaheurystyki:

- **Bezwzględna dopuszczalność rozwiązań:** Zastosowanie procedury `napraw()` pozwoliło na uzyskanie bezkonfliktowych macierzy wyrównań, co stanowi fundament poprawności biologicznej oraz rozwiązania problemu.
- **Wysoka odporność na szum (Mutacje):** Algorytm wykazał się dużą stabilnością na zanieczyszczenia danych (Mutacje). Różnice w osiąganej wartości funkcji celu dla czystych danych (brak mutacji) oraz mocno zaszumionych (15/30 mutacji) były niewielkie. Wynika to z faktu, iż lokalne operatory przeszukiwania efektywnie izolują zmutowane znaki, otaczając je lukami, co zapobiega powstawaniu konfliktów w kluczowych kolumnach.
- **Wydajność Równoległego Multi-Start:** Równoległe uruchomienie kilku niezależnych wątków z różnymi ziarnami (ang. *seed*) losowości (*seed*) pozwoliło na szeroką eksplorację przestrzeni rozwiązań. Nawet jeśli jeden z wątków utknął w trudnym minimum lokalnym, pozostałe mogły znaleźć lepsze dopasowanie globalne, co znacząco zwiększyło powtarzalność otrzymywanych wyników.

Wady i ograniczenia:

Jednakże eksperymenty ze skalowalnością (dla dużych i gęstych instancji, np. $m = 15, n = 70$) jasno udowodniły NP-trudną naturę problemu. Im więcej sekwencji, tym bardziej funkcja celu (długość wyrównania) pogarszała się w porównaniu do długości nadciągu bazowego d . W zbyt zwartej macierzy każdy ruch kompresujący napędza duży napływ konfliktów w innych kolumnach i tak na przemian, aż algorytm utyka z operatorami sąsiedztwa na barierze nie do przejścia.

8.2 Perspektywy rozwoju algorytmu

Choć obecna implementacja spełnia swoje zadanie, w celu adaptacji oprogramowania do przemysłowych lub laboratoryjnych instancji bioinformatycznych (np. dopasowywania setek długich sekwencji DNA), konieczne byłoby wprowadzenie następujących optymalizacji:

1. **Parametryzacja dywersyfikacji:** Zamiast standardowych wartości działających jak mechanizm (*Heartbeat* pulsowanie wag z 2000 na 1 co 100 iteracji), można wdrożyć mechanizm reaktywny. Algorytm mógłby dynamicznie analizować poziom stagnacji i płynnie modyfikować wagi kary lub parametry *Tabu Tenure* w zależności od gęstości eksplorowanego obszaru.
2. **Równoległość na poziomie GPU (CUDA/OpenCL):** Złożoność tej części algorytmu wynosi $\mathcal{O}(I \cdot k \cdot m \cdot L)$ i głównie wiąże się z liczeniem głosów w kolumnach (czyli wykonywaniem funkcji *oceniaj*). Ponieważ podczas oceny kolumny są niezależne od siebie, to tę część można w bardzo łatwy sposób zrównoleglić na setkach rdzeni kart graficznych i dzięki temu zyskać kilka rzędów wielkości w czasie potrzebnym do ewaluacji sąsiadów.
3. **Algorytmy memetyczne (połączenie z GA):** pojedyncza trajektoria przeszukiwania z tabu radzi sobie słabo z szerokimi barierami błędów. Ciekawe byłoby więc połączenie go z algorytmem populacyjnym, np. Algorytmem Genetycznym. AG częściowo poradziłby sobie z takimi barierami (krzyżując całe macierze), natomiast przeszukiwanie z tabu pełniłoby rolę wysoce wyspecjalizowanego optymalizatora lokalnego dla poszczególnych rozwiązań (tak więc powstałby algorytm memetyczny).
4. **Ergonomia i analityka w GUI:** W warstwie interfejsu warto byłoby również dodać możliwość podglądu macierzy na żywo, który kolorowałby kolumny na czerwono w miejscach konfliktów. Dodatkowo, funkcja eksportu powinna zapisywać wyrównanie w standardowych formatach bioinformatycznych (np. FASTA lub CLUSTAL), co umożliwiłoby integrację narzędzia z zewnętrznymi pakietami analitycznymi.